

Graph

Counting Rooms

```
#include

using namespace std;

char c;
int N, M, cnt;
int h[] = {1, -1, 0, 0}, v[] = {0, 0, 1, -1};
bool vis[1000][1000];

void dfs(int x, int y){
    vis[x][y] = true;
    for(int i = 0; i < 4; i++){
        int dx = x+h[i], dy = y+v[i];
        if(0 <= dx && dx < N && 0 <= dy && dy < M && !vis[dx][dy])
            dfs(dx, dy);
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++){
        for(int j = 0; j < M; j++){
            scanf(" %c", &c);
            vis[i][j] = (c == '#');
        }
    }
    for(int i = 0; i < N; i++){
        for(int j = 0; j < M; j++){
            if(!vis[i][j]){
                dfs(i, j);
                cnt++;
            }
        }
    }
    printf("%d\n", cnt);
}
```

Labyrinth

```
#include

using namespace std;
typedef pair pii;
#define x first
#define y second

const int h[] = {1, -1, 0, 0}, v[] = {0, 0, 1, -1};

bool vis[1000][1000];
char c, par[1000][1000], ans[1000000];
```

```

int N, M, sx, sy, ex, ey, dist[1000][1000];
queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++){
        for(int j = 0; j < M; j++){
            scanf(" %c", &c);
            if(c == '#') vis[i][j] = true;
            else if(c == 'A'){
                sx = i; sy = j;
            } else if(c == 'B'){
                ex = i; ey = j;
            }
        }
    }

    vis[sx][sy] = true;
    Q.push({sx, sy});
    while(!Q.empty()){
        pii P = Q.front(); Q.pop();
        for(int i = 0; i < 4; i++){
            int dx = P.x+h[i];
            int dy = P.y+v[i];
            if(0 <= dx && dx < N && 0 <= dy && dy < M && !vis[dx][dy]){
                if(i == 0) par[dx][dy] = 'D';
                else if(i == 1) par[dx][dy] = 'U';
                else if(i == 2) par[dx][dy] = 'R';
                else if(i == 3) par[dx][dy] = 'L';
                dist[dx][dy] = dist[P.x][P.y]+1;
                vis[dx][dy] = true;
                Q.push({dx, dy});
            }
        }
    }

    if(!vis[ex][ey]){
        printf("NO\n");
        return 0;
    }

    printf("YES\n%d\n", dist[ex][ey]);
    pii P = {ex, ey};
    for(int i = dist[ex][ey]; i > 0; i--){
        ans[i] = par[P.x][P.y];
        if(ans[i] == 'D') P = {P.x-1, P.y};
        else if(ans[i] == 'U') P = {P.x+1, P.y};
        else if(ans[i] == 'R') P = {P.x, P.y-1};
        else if(ans[i] == 'L') P = {P.x, P.y+1};
    }
    for(int i = 1; i <= dist[ex][ey]; i++)
        printf("%c", ans[i]);
    printf("\n");
}

```

Building Roads

```

#include

using namespace std;

```

```

typedef pair pii;

int N, M, a, b, ds[100001];
vector ans;

int find(int u){
    if(ds[u] < 0)    return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v)    return false;
    if(ds[u] < ds[v])    swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++) ds[i] = -1;
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        merge(a, b);
    }

    for(int i = 1; i < N; i++)
        if(merge(i, i+1))
            ans.push_back({i, i+1});

    printf("%d\n", (int) ans.size());
    for(pii P : ans)
        printf("%d %d\n", P.first, P.second);
}

```

Message Route

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, K, a, b, p[maxN], dist[maxN];
bool vis[maxN];
vector G[maxN];
queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    Q.push(1);
    vis[1] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();

```

```

        for(int v : G[u]){
            if(!vis[v]){
                dist[v] = dist[u]+1;
                vis[v] = true;
                p[v] = u;
                Q.push(v);
            }
        }
    }

    if(!vis[N]){
        printf("IMPOSSIBLE\n");
        return 0;
    }

    int u = N;
    K = dist[N];
    vector ans(K+1);
    for(int i = K; i >= 0; i--){
        ans[i] = u;
        u = p[u];
    }

    printf("%d\n", K+1);
    for(int i = 0; i <= K; i++){
        printf("%d%c", ans[i], (" \n")[i==K]);
    }
}

```

Building Teams

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, a, b;
bool possible, vis[maxN], team[maxN];
vector G[maxN];

void dfs(int u, int p = 0){
    for(int v : G[u]){
        if(v != p){
            if(!vis[v]){
                team[v] = !team[u];
                vis[v] = true;
                dfs(v, u);
            } else
                if(team[v] == team[u])
                    possible = false;
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }
}

```

```

possible = true;
for(int i = 1; i <= N; i++){
    if(!vis[i]){
        vis[i] = true;
        dfs(i);
    }
}

if(!possible)
    printf("IMPOSSIBLE\n");
else
    for(int i = 1; i <= N; i++)
        printf("%d%c", (team[i] ? 1 : 2), (" \n")[i==N]);
}

```

Round Trip

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, a, b, p[maxN], ds[maxN];
vector ans, G[maxN];

void dfs(int u){
    for(int v : G[u]){
        if(v != p[u]){
            p[v] = u;
            dfs(v);
        }
    }
}

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    fill(ds+1, ds+N+1, -1);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        if(!merge(a, b)){
            dfs(a);

            int u = b;
            while(u != 0){
                ans.push_back(u);
                u = p[u];
            }
        }
    }
}

```

```

    }

    int K = ans.size();
    printf("%d\n", K+1);
    for(int j = 0; j < K; j++)
        printf("%d ", ans[j]);
    printf("%d\n", b);

    return 0;
} else {
    G[a].push_back(b);
    G[b].push_back(a);
}
}
printf("IMPOSSIBLE\n");
}

```

Monsters

```

#include

using namespace std;
const int maxN = 1000;
const int INF = 0x3f3f3f3f;

int h[] = {1, -1, 0, 0};
int v[] = {0, 0, 1, -1};
struct point { int x, y; };

int N, M, sx, sy;
bool vis[maxN][maxN];
char ans[maxN*maxN], c[maxN][maxN], p[maxN][maxN];
int d1[maxN][maxN], d2[maxN][maxN];
queue Q;

bool inbounds(int x, int y){
    return 0 <= x && x < N && 0 <= y && y < M;
}

void printsolution(int x, int y){
    int D = d2[x][y];
    printf("YES\n%d\n", D);
    for(int i = D-1; i >= 0; i--){
        ans[i] = p[x][y];
        if(ans[i] == 'D')    x--;
        else if(ans[i] == 'U')    x++;
        else if(ans[i] == 'R')    y--;
        else if(ans[i] == 'L')    y++;
    }
    for(int i = 0; i < D; i++)
        printf("%c", ans[i]);
    printf("\n");
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++){
        for(int j = 0; j < M; j++){
            scanf(" %c", &c[i][j]);
            if(c[i][j] == '#')    vis[i][j] = true;
            else if(c[i][j] == 'M'){

```

```

        vis[i][j] = true;
        Q.push({i, j});
    } else if(c[i][j] == 'A'){
        sx = i; sy = j;
    }
}

while(!Q.empty()){
    point P = Q.front(); Q.pop();
    for(int i = 0; i < 4; i++){
        int x = P.x + h[i];
        int y = P.y + v[i];
        if(inbounds(x, y) && !vis[x][y]){
            d1[x][y] = d1[P.x][P.y] + 1;
            vis[x][y] = true;
            Q.push({x, y});
        }
    }
}

for(int i = 0; i < N; i++){
    for(int j = 0; j < M; j++){
        if(!vis[i][j]) d1[i][j] = INF;
        vis[i][j] = (c[i][j] == '#');
    }
}

vis[sx][sy] = true;
Q.push({sx, sy});
while(!Q.empty()){
    point P = Q.front(); Q.pop();
    for(int i = 0; i < 4; i++){
        int x = P.x + h[i];
        int y = P.y + v[i];
        if(inbounds(x, y) && !vis[x][y] && d2[P.x][P.y]+1 < d1[x][y]){
            if(i == 0) p[x][y] = 'D';
            else if(i == 1) p[x][y] = 'U';
            else if(i == 2) p[x][y] = 'R';
            else if(i == 3) p[x][y] = 'L';
            d2[x][y] = d2[P.x][P.y] + 1;
            vis[x][y] = true;
            Q.push({x, y});
        }
    }
}

for(int i = 0; i < N; i++){
    if(c[i][0] != '#' && c[i][0] != 'M' && vis[i][0]){
        printsolution(i, 0);
        return 0;
    } else if(c[i][M-1] != '#' && c[i][M-1] != 'M' && vis[i][M-1]){
        printsolution(i, M-1);
        return 0;
    }
}

for(int i = 0; i < M; i++){
    if(c[0][i] != '#' && c[0][i] != 'M' && vis[0][i]){
        printsolution(0, i);
        return 0;
    } else if(c[N-1][i] != '#' && c[N-1][i] != 'M' && vis[N-1][i]){
        printsolution(N-1, i);
        return 0;
    }
}

printf("NO\n");
}

```

Shortest Routes I

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e5;

struct Edge { int v; ll w; };
struct Node {
    int id; ll dist;
    friend bool operator<(const Node &a, const Node &b){
        return a.dist > b.dist;
    }
};

int N, M, a, b;
ll dist[maxN+1], c;
vector G[maxN+1];
priority_queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d %lld", &a, &b, &c);
        G[a].push_back({b, c});
    }

    memset(dist, 0x3f, sizeof(dist));
    dist[1] = 0;
    Q.push({1, 0});
    while(!Q.empty()){
        ll d = Q.top().dist;
        int u = Q.top().id;
        Q.pop();

        if(d > dist[u]) continue;

        for(Edge e : G[u]){
            if(dist[e.v] > d+e.w){
                dist[e.v] = d+e.w;
                Q.push({e.v, d+e.w});
            }
        }
    }

    for(int i = 1; i <= N; i++)
        printf("%lld%c", dist[i], (" \n")[i==N]);
}
```

Shortest Routes II

```
#include

using namespace std;
```

```

typedef long long ll;
const int maxN = 500;
const ll INF = 0x3f3f3f3f3f3f3f3f;

int N, M, Q, a, b;
ll dist[maxN+1][maxN+1], c;

int main(){
    scanf("%d %d %d", &N, &M, &Q);
    memset(dist, 0x3f, sizeof(dist));
    for(int i = 0; i < M; i++){
        scanf("%d %d %lld", &a, &b, &c);
        dist[a][b] = min(dist[a][b], c);
        dist[b][a] = min(dist[b][a], c);
    }

    for(int i = 1; i <= N; i++) dist[i][i] = 0;
    for(int k = 1; k <= N; k++)
        for(int i = 1; i <= N; i++)
            for(int j = 1; j <= N; j++)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

    for(int q = 0; q < Q; q++){
        scanf("%d %d", &a, &b);
        printf("%lld\n", (dist[a][b] >= INF ? -1 : dist[a][b]));
    }
}

```

High Score

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2501;
const int maxM = 5001;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Edge {
    int a, b; ll c;
} edges[maxM];

int N, M;
ll dp[maxN];
bool vis[maxN], visR[maxN];
vector G[maxN], GR[maxN];

void dfs(int u){
    vis[u] = true;
    for(int v : G[u])
        if(!vis[v])
            dfs(v);
}

void dfsR(int u){
    visR[u] = true;
    for(int v : GR[u])
        if(!visR[v])
            dfsR(v);
}

```

```

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        ll c;
        scanf("%d %d %lld", &a, &b, &c);
        edges[i] = {a, b, -c};
        G[a].push_back(b);
        GR[b].push_back(a);
    }
    dfs(1); dfsR(N);

    fill(dp+2, dp+N+1, INF);
    bool improvement = true;
    for(int iter = 0; iter < N && improvement; iter++){
        improvement = false;
        for(int i = 0; i < M; i++){
            int u = edges[i].a;
            int v = edges[i].b;
            ll w = edges[i].c;

            if(dp[v] > dp[u]+w){
                dp[v] = dp[u]+w;
                improvement = true;

                if(iter == N-1 && vis[v] && visR[v]){
                    printf("-1\n");
                    return 0;
                }
            }
        }
    }

    printf("%lld\n", -dp[N]);
}

```

Flight Discount

```

#include

using namespace std;
typedef long long ll;
typedef pair edge;
typedef tuple node;
const int maxN = 2e5+1;

int N, M, a, b;
ll c, dist[2][maxN];
vector G[maxN];
priority_queue, greater> Q;

int main(){
    scanf("%d %d", &N, &M);
    memset(dist, 0x3f, sizeof(dist));
    for(int i = 0; i < M; i++){
        scanf("%d %d %lld", &a, &b, &c);
        G[a].push_back({b, c});
    }

    dist[0][1] = dist[1][1] = 0;
    Q.push({0, 1, 1});
    while(!Q.empty()){

```

```

ll d = get<0>(Q.top());
int u = get<1>(Q.top());
int coupon = get<2>(Q.top());
Q.pop();

if(dist[!coupon][u] < d)    continue;

for(edge e : G[u]){
    int v = e.first;
    ll w = e.second;

    if(coupon){
        if(dist[0][v] > d + w){
            dist[0][v] = d + w;
            Q.push({d+w, v, 1});
        }
        if(dist[1][v] > d + w/2){
            dist[1][v] = d + w/2;
            Q.push({d+w/2, v, 0});
        }
    } else {
        if(dist[1][v] > d + w){
            dist[1][v] = d + w;
            Q.push({d+w, v, 0});
        }
    }
}
}

printf("%lld\n", min(dist[0][N], dist[1][N]));
}

```

Cycle Finding

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2501;
const int maxM = 5001;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Edge {
    int a, b; ll c;
} edges[maxM];

int N, M, ptr, p[maxN];
ll dp[maxN];

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        ll c;
        scanf("%d %d %lld", &a, &b, &c);
        edges[i] = {a, b, c};
    }

    ptr = -1;
    fill(dp+2, dp+N+1, INF);
    for(int iter = 0; iter < N && ptr; iter++){
        ptr = 0;

```

```

    for(int i = 0; i < M; i++){
        int u = edges[i].a;
        int v = edges[i].b;
        ll w = edges[i].c;

        if(dp[v] > dp[u]+w){
            dp[v] = dp[u]+w;
            p[v] = u;
            ptr = v;
        }
    }
}

if(!ptr){
    printf("NO\n");
    return 0;
}

for(int i = 0; i < N; i++){
    ptr = p[ptr];

    vector cycle;
    for(int v = ptr;; v = p[v]){
        cycle.push_back(v);
        if(v == ptr && (int) cycle.size() > 1)
            break;
    }
    reverse(cycle.begin(), cycle.end());

    printf("YES\n");
    int K = (int) cycle.size();
    for(int i = 0; i < K; i++)
        printf("%d%c", cycle[i], (" \n")[i==K-1]);
}

```

Flight Routes

```

#include

using namespace std;
typedef long long ll;
typedef pair edge;
typedef pair node;
const int maxN = 2e5+1;

int N, M, K, cnt[maxN];
vector G[maxN];
priority_queue, greater> Q;

int main(){
    scanf("%d %d %d", &N, &M, &K);
    for(int i = 0, a, b, c; i < M; i++){
        scanf("%d %d %d", &a, &b, &c);
        G[a].push_back({b, c});
    }

    Q.push({0, 1});
    while(!Q.empty()){
        ll d = get<0>(Q.top());
        int u = get<1>(Q.top());
        Q.pop();

```

```

        cnt[u]++;
        if(u == N){
            printf("%lld%c", d, (" \n")[cnt[u]==K]);
            if(cnt[u] == K) return 0;
        }

        if(cnt[u] <= K){
            for(edge e : G[u]){
                int v = e.first;
                ll w = e.second;
                Q.push({d+w, v});
            }
        }
    }
}

```

Round Trip II

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, start, finish, p[maxN], vis[maxN];
vector ans, G[maxN];

void dfs(int u){
    vis[u] = 1;
    for(int v : G[u]){
        if(vis[v] == 0){
            p[v] = u;
            dfs(v);
            if(start)
                return;
        } else if(vis[v] == 1){
            finish = u;
            start = v;
            return;
        }
    }
    vis[u] = 2;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
    }

    for(int i = 1; i <= N && !start; i++)
        if(vis[i] == 0)
            dfs(i);

    if(!start){
        printf("IMPOSSIBLE\n");
        return 0;
    }

    ans.push_back(start);

```

```

    for(int u = finish; u != start; u = p[u])
        ans.push_back(u);
    ans.push_back(start);
    reverse(ans.begin(), ans.end());

    int K = (int) ans.size();
    printf("%d\n", K);
    for(int i = 0; i < K; i++)
        printf("%d%c", ans[i], (" \n")[i==K-1]);
}

```

Course Schedule

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, a, b, cnt, in[maxN];
vector ans, G[maxN];
queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        in[b]++;
    }

    for(int i = 1; i <= N; i++)
        if(in[i] == 0)
            Q.push(i);

    cnt = N;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        ans.push_back(u);
        cnt--;

        for(int v : G[u]){
            in[v]--;
            if(in[v] == 0)
                Q.push(v);
        }
    }

    if(cnt)
        printf("IMPOSSIBLE\n");
    else
        for(int i = 0; i < N; i++)
            printf("%d%c", ans[i], (" \n")[i==N-1]);
}

```

Longest Flight Route

```
#include

using namespace std;
const int maxN = 1e5+1;

bool vis[maxN];
int N, M, K, a, b, in[maxN], p[maxN], l[maxN], ans[maxN];
vector G[maxN];
queue Q;

void dfs(int u, int par = 0){
    vis[u] = true;
    for(int v : G[u])
        if(v != par && !vis[v])
            dfs(v, u);
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        in[b]++;
    }

    dfs(1);
    if(!vis[N]){
        printf("IMPOSSIBLE\n");
        return 0;
    }

    fill(l+2, l+maxN, -1);
    for(int i = 1; i <= N; i++)
        if(in[i] == 0)
            Q.push(i);

    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        for(int v : G[u]){
            if(l[u] != -1 && l[v] < l[u]+1){
                l[v] = l[u] + 1;
                p[v] = u;
            }
            in[v]--;
            if(in[v] == 0)
                Q.push(v);
        }
    }

    K = l[N] - l[1];
    printf("%d\n", K+1);
    for(int i = K, u = N; i >= 0; i--){
        ans[i] = u;
        u = p[u];
    }
    for(int i = 0; i <= K; i++)
        printf("%d%c", ans[i], (" \n")[i==K]);
}
```

Game Routes

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+1;
const ll MOD = 1e9+7;

int N, M, a, b, cnt, in[maxN];
ll dp[maxN];
vector G[maxN];
queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        in[b]++;
    }

    for(int i = 1; i <= N; i++)
        if(in[i] == 0)
            Q.push(i);

    dp[1] = 1;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        for(int v : G[u]){
            dp[v] = (dp[v] + dp[u]) % MOD;
            in[v]--;
            if(in[v] == 0)
                Q.push(v);
        }
    }

    printf("%lld\n", dp[N]);
}
```

Investigation

```
#include

using namespace std;
typedef long long ll;
typedef pair edge;
typedef pair node;
const int maxN = 1e5+1;
const ll MOD = 1e9+7;
const ll INF = 0x3f3f3f3f3f3f;

int N, M, a, b, minR[maxN], maxR[maxN];
ll c, dist[maxN], ways[maxN];
vector G[maxN];

struct Comparator {
    bool operator() (int x, int y) const {
        return dist[x] == dist[y] ? x < y : dist[x] < dist[y];
    }
}
```

```

    }
};
set PQ;

int main(){
    scanf("%d %d", &N, &M);
    memset(dist, 0x3f, sizeof(dist));
    for(int i = 0; i < M; i++){
        scanf("%d %d %lld", &a, &b, &c);
        G[a].push_back({b, c});
    }

    PQ.insert(1);
    ways[1] = 1;
    dist[1] = 0;
    while(!PQ.empty()){
        int u = *PQ.begin();
        ll d = dist[u];
        PQ.erase(PQ.begin());

        for(edge e : G[u]){
            int v = e.first;
            ll w = e.second;
            if(d + w <= dist[v]){
                if(dist[v] != INF) PQ.erase(v);
                if(d + w == dist[v]){
                    ways[v] = (ways[u] + ways[v]) % MOD;
                    minR[v] = min(minR[v], minR[u]+1);
                    maxR[v] = max(maxR[v], maxR[u]+1);
                }
                if(d + w < dist[v]){
                    dist[v] = d+w;
                    ways[v] = ways[u];
                    minR[v] = minR[u]+1;
                    maxR[v] = maxR[u]+1;
                }
                PQ.insert(v);
            }
        }
    }

    printf("%lld %lld %d %d\n", dist[N], ways[N], minR[N], maxR[N]);
}

```

Planets Queries I

```

#include

using namespace std;
const int maxN = 2e5+1;
const int logK = 31;

int N, Q, x, k, p[maxN][logK];

int main(){
    scanf("%d %d", &N, &Q);
    for(int i = 1; i <= N; i++)
        scanf("%d", &p[i][0]);
    for(int j = 1; j < logK; j++)
        for(int i = 1; i <= N; i++)
            p[i][j] = p[p[i][j-1]][j-1];
}

```

```

for(int q = 0; q < Q; q++){
    scanf("%d %d", &x, &k);
    for(int i = 0; i < logK; i++)
        if(k&(1<

```

Planets Queries II

```

#include

using namespace std;
const int maxN = 2e5+1, logN = 20;

bool vis[maxN];
int N, Q, ds[maxN], d[maxN], comp[maxN], cyc[maxN], p[logN][maxN];
vector cycleSeeds, G[maxN];

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

void init_parents(){
    for(int i = 1; i < logN; i++)
        for(int j = 1; j <= N; j++)
            p[i][j] = p[i-1][p[i-1][j]];
}

int travel(int u, int k){
    int v = u;
    for(int i = logN-1; i >= 0; i--){
        if(k&(1<= 0; i--){
            int par = p[i][root];
            if(!cyc[par])
                root = par;
        }
    }
    root = p[0][root];

    return (d[a]-d[root]) + query(root, b);
}

int main(){
    scanf("%d %d", &N, &Q);
    fill(ds+1, ds+N+1, -1);
    for(int i = 1, x; i <= N; i++){
        scanf("%d", &x);
        p[0][i] = x;
        G[x].push_back(i);
        if(!merge(x, i))
            cycleSeeds.push_back(x);
    }

```

```

init_parents();
int compID = 1;
for(int seed : cycleSeeds)
    dfs(seed, compID++);

for(int i = 0, a, b; i < Q; i++){
    scanf("%d %d", &a, &b);
    printf("%d\n", query(a, b));
}
}

```

Planets Cycles

```

#include

using namespace std;
const int maxN = 2e5+1;

bool vis[maxN];
int N, t[maxN], in[maxN], ans[maxN];
vector G[maxN];
queue Q;

void dfs1(int u){
    for(int v : G[u]){
        if(!vis[v]){
            ans[v] = ans[u]+1;
            vis[v] = true;
            dfs1(v);
        }
    }
}

void dfs2(int u, int d = 1){
    vis[u] = true;
    int v = t[u];
    if(vis[v]){
        ans[u] = d;
    } else {
        dfs2(v, d+1);
        ans[u] = ans[v];
    }
    dfs1(u);
}

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d", &t[i]);
        G[t[i]].push_back(i);
        in[t[i]]++;
    }

    for(int i = 1; i <= N; i++)
        if(in[i] == 0)
            Q.push(i);

    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        int v = t[u];
    }
}

```

```

        in[v]--;
        if(in[v] == 0)
            Q.push(v);
    }

    for(int i = 1; i <= N; i++)
        if(in[i] && !vis[i])
            dfs2(i);

    for(int i = 1; i <= N; i++)
        printf("%d%c", ans[i], (" \n")[i==N]);
}

```

Road Reparation

```

#include

using namespace std;
#define f first
#define s second
typedef pair pii;
typedef pair edge;
const int maxN = 1e5+1;
const int maxM = 2e5+1;

int N, M, a, b, c, ds[maxN];
long long sum;
edge edges[maxM];

int find(int u){
    if(ds[u] < 0)    return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v)    return false;
    if(ds[u] < ds[v])    swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    fill(ds+1, ds+N+1, -1);
    for(int i = 0; i < M; i++){
        scanf("%d %d %d", &a, &b, &c);
        edges[i] = {c, {a, b}};
    }
    sort(edges, edges+M);

    for(int i = 0; i < M; i++){
        edge e = edges[i];
        if(merge(e.s.f, e.s.s)){
            sum += e.f;
            N--;
        }
    }
}

```

```
    if(N == 1) printf("%lld\n", sum);
    else      printf("IMPOSSIBLE\n");
}
```

Road Construction

```
#include

using namespace std;
const int maxN = 1e5;

int N, M, a, b, components, largest, ds[maxN+1];

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    fill(ds+1, ds+N+1, -1);

    components = N;
    largest = 1;
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        if(merge(a, b)){
            components--;
            largest = max(largest, -ds[find(a)]);
        }
        printf("%d %d\n", components, largest);
    }
}
```

Flight Routes Check

```
#include

using namespace std;
const int maxN = 2e5+1;

int N, M, a, b;
```

```

bool vis1[maxN], vis2[maxN];
vector G1[maxN], G2[maxN];

void dfs1(int u = 1, int p = 0){
    vis1[u] = true;
    for(int v : G1[u])
        if(v != p && !vis1[v])
            dfs1(v, u);
}

void dfs2(int u = 1, int p = 0){
    vis2[u] = true;
    for(int v : G2[u])
        if(v != p && !vis2[v])
            dfs2(v, u);
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G1[a].push_back(b);
        G2[b].push_back(a);
    }
    dfs1();
    dfs2();
    for(int i = 1; i <= N; i++){
        if(!vis1[i] || !vis2[i]){
            printf("NO\n");
            if(!vis1[i])    printf("1 %d\n", i);
            else            printf("%d 1\n", i);
            return 0;
        }
    }
    printf("YES\n");
}

```

Planets and Kingdoms

```

#include

using namespace std;
const int maxN = 1e5+1;

bool vis[maxN];
int N, M, rt[maxN];
vector ord, comp, G[maxN], GR[maxN];

void dfs1(int u){
    vis[u] = true;
    for(int v : G[u])
        if(!vis[v])
            dfs1(v);
    ord.push_back(u);
}

void dfs2(int u){
    vis[u] = true;
    comp.push_back(u);
    for(int v : GR[u])
        if(!vis[v])

```

```

        dfs2(v);
    }

    int main(){
        scanf("%d %d", &N, &M);
        for(int i = 0, a, b; i < M; i++){
            scanf("%d %d", &a, &b);
            G[a].push_back(b);
            GR[b].push_back(a);
        }

        for(int i = 1; i <= N; i++)
            if(!vis[i])
                dfs1(i);

        int K = 0;
        fill(vis+1, vis+N+1, false);
        reverse(ord.begin(), ord.end());
        for(int u : ord){
            if(!vis[u]){
                dfs2(u);
                K++;
                for(int v : comp)
                    rt[v] = K;
                comp.clear();
            }
        }

        printf("%d\n", K);
        for(int i = 1; i <= N; i++)
            printf("%d%c", rt[i], (" \n")[i==N]);
    }

```

Giant Pizza

```

#include

using namespace std;
const int maxN = 2e5+1;

bool vis[maxN];
char ans[maxN];
int N, M, K, in[maxN], rt[maxN];
vector ord, comp, G[maxN], GR[maxN], C[maxN], SCC[maxN];

int flip(int x){
    return (x&1 ? x+1 : x-1);
}

void add_edge(char c1, int a, char c2, int b){
    a = 2*a - (c1 == '-');
    b = 2*b - (c2 == '-');
    G[flip(a)].push_back(b);
    G[flip(b)].push_back(a);
    GR[a].push_back(flip(b));
    GR[b].push_back(flip(a));
}

void dfs1(int u){
    vis[u] = true;
    for(int v : G[u])

```

```

        if(!vis[v])
            dfs1(v);
    ord.push_back(u);
}

void dfs2(int u){
    vis[u] = true;
    comp.push_back(u);
    for(int v : GR[u])
        if(!vis[v])
            dfs2(v);
}

int main(){
    scanf("%d %d", &M, &N);
    for(int i = 0, a, b; i < M; i++){
        char c1, c2;
        scanf(" %c %d %c %d", &c1, &a, &c2, &b);
        add_edge(c1, a, c2, b);
    }

    for(int i = 1; i <= 2*N; i++)
        if(!vis[i])
            dfs1(i);

    fill(vis+1, vis+2*N+1, false);
    reverse(ord.begin(), ord.end());
    for(int u : ord){
        if(!vis[u]){
            dfs2(u);
            K++;
            for(int v : comp){
                rt[v] = K;
                C[K].push_back(v);
            }
            comp.clear();
        }
    }

    // Impossible iff x and not(x) belong to same SCC
    for(int i = 1; i <= N; i++){
        if(rt[2*i] == rt[2*i-1]){
            printf("IMPOSSIBLE\n");
            return 0;
        }
    }

    for(int u = 1; u <= 2*N; u++){
        for(int v : G[u]){
            if(rt[u] != rt[v]){
                SCC[rt[u]].push_back(rt[v]);
                in[rt[v]]++;
            }
        }
    }

    queue Q;
    ord.clear();
    for(int u = 1; u <= K; u++){
        if(in[u] == 0){
            ord.push_back(u);
            Q.push(u);
        }
    }

    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        for(int v : SCC[u]){
            in[v]--;
            if(in[v] == 0){

```

```

        ord.push_back(v);
        Q.push(v);
    }
}

fill(vis+1, vis+N+1, false);
reverse(ord.begin(), ord.end());
for(int k : ord){
    for(int u : C[k]){
        int i = (u+1)/2;
        if(!vis[i]){
            ans[i] = (u&1 ? '-' : '+');
            vis[i] = true;
        }
    }
}

for(int i = 1; i <= N; i++)
    printf("%c%c", ans[i], (" \n")[i==N]);
}

```

Coin Collector

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+1;

bool vis[maxN];
int N, M, K, rt[maxN];
ll k[maxN], ck[maxN], dp[maxN];
vector ord, comp, G[maxN], GR[maxN], SCC[maxN];

void dfs1(int u){
    vis[u] = true;
    for(int v : G[u])
        if(!vis[v])
            dfs1(v);
    ord.push_back(u);
}

void dfs2(int u){
    vis[u] = true;
    comp.push_back(u);
    for(int v : GR[u])
        if(!vis[v])
            dfs2(v);
}

void dfs3(int u){
    vis[u] = true;
    dp[u] = ck[u];
    for(int v : SCC[u]){
        if(!vis[v])
            dfs3(v);
        dp[u] = max(dp[u], dp[v]+ck[u]);
    }
}

```

```

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++){
        scanf("%d", &k[i]);

        for(int i = 0, a, b; i < M; i++){
            scanf("%d %d", &a, &b);
            G[a].push_back(b);
            GR[b].push_back(a);
        }

        for(int i = 1; i <= N; i++){
            if(!vis[i])
                dfs1(i);

            fill(vis+1, vis+N+1, false);
            reverse(ord.begin(), ord.end());
            for(int u : ord){
                if(!vis[u]){
                    dfs2(u);
                    K++;
                    for(int v : comp){
                        ck[K] += k[v];
                        rt[v] = K;
                    }
                    comp.clear();
                }
            }

            for(int u = 1; u <= N; u++){
                for(int v : G[u])
                    if(rt[v] != rt[u])
                        SCC[rt[u]].push_back(rt[v]);

                fill(vis+1, vis+K+1, false);
                for(int i = 1; i <= K; i++){
                    if(!vis[i])
                        dfs3(i);

                    ll best = 0;
                    for(int i = 1; i <= K; i++){
                        best = max(best, dp[i]);
                    }
                    printf("%lld\n", best);
                }
            }
        }
    }
}

```

Mail Delivery

```

#include

using namespace std;
typedef pair pii;
const int maxN = 1e5+1;
const int maxM = 2e5+1;

int N, M, deg[maxN];
bool tour_exists, used[maxM];
vector G[maxN];
vector tour;
stack S;

int main(){

```

```

scanf("%d %d", &N, &M);
for(int i = 0, a, b; i < M; i++){
    scanf("%d %d", &a, &b);
    G[a].push_back({b, i});
    G[b].push_back({a, i});
    deg[a]++; deg[b]++;
}

tour_exists = true;
for(int i = 1; i <= N; i++)
    if(deg[i]&1)
        tour_exists = false;

if(!tour_exists){
    printf("IMPOSSIBLE\n");
    return 0;
}

S.push(1);
while(!S.empty()){
    int u = S.top();
    if(deg[u]){
        while(!G[u].empty()){
            int v = G[u].back().first;
            int id = G[u].back().second;
            G[u].pop_back();

            if(!used[id]){
                deg[u]--; deg[v]--;
                used[id] = true;
                S.push(v);
                break;
            }
        }
    }
    else {
        tour.push_back(u);
        S.pop();
    }
}

// Should be M+1 nodes in the Euler tour
// If not, it means the graph was not connected
if((int) tour.size() != M+1)
    printf("IMPOSSIBLE\n");
else
    for(int i = 0; i <= M; i++)
        printf("%d%c", tour[i], (" \n")[i==M]);
}

```

De Bruijn Sequence

```

#include

using namespace std;
const int maxN = 15;
const int SIZE = (1< G[SIZE];
stack S;

void init(){
    for(int u = 0; u < 1<<(N-1); u++){
        int v = (u<<1)%(1<<(N-1));

```

```

        G[u].push_back(v);
        G[u].push_back(v|1);
    }
}

void solve(){
    S.push(0);
    while(!S.empty()){
        int u = S.top();
        if(!G[u].empty()){
            int v = G[u].back();
            G[u].pop_back();
            S.push(v);
        } else {
            ans[ansptr++] = (char) (u&1) + '0';
            S.pop();
        }
    }
    for(int i = 0; i < N-2; i++)
        ans[ansptr++] = '0';
}

int main(){
    scanf("%d", &N);
    if(N == 1){
        printf("01");
        return 0;
    }

    init();
    solve();
    for(int i = 0; i < ansptr; i++)
        printf("%c", ans[i]);
}

```

Teleporters Path

```

#include

using namespace std;
const int maxN = 1e5+1;

bool vis[maxN];
int N, M, in[maxN];
vector tour, G[maxN];
stack S;

void dfs(int u = 1, int p = -1){
    vis[u] = true;
    for(int v : G[u])
        if(v != p && !vis[v])
            dfs(v, u);
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        in[b]++;
    }
}

```

```

dfs();
bool exists = vis[N];
exists &= (in[1]+1 == (int) G[1].size());
exists &= (in[N]-1 == (int) G[N].size());
for(int i = 2; i < N; i++){
    exists &= (in[i] == (int) G[i].size());
    if(!vis[i]) exists &= (in[i] == 0 && G[i].size() == 0);
}

if(!exists){
    printf("IMPOSSIBLE\n");
    return 0;
}

S.push(1);
while(!S.empty()){
    int u = S.top();
    if(!G[u].empty()){
        int v = G[u].back();
        G[u].pop_back();
        S.push(v);
    } else {
        tour.push_back(u);
        S.pop();
    }
}

reverse(tour.begin(), tour.end());
for(int i = 0; i < M+1; i++)
    printf("%d%c", tour[i], (" \n")[i==M]);
}

```

Hamiltonian Flights

```

#include

using namespace std;
typedef long long ll;
typedef pair pii;
const int maxN = 20;
const int SIZE = 1< G[maxN];
queue Q;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a-1].push_back(b-1);
    }

    dp[0][1] = 1;
    Q.push({0, 1});
    inqueue[0][1] = true;
    while(!Q.empty()){
        int u = Q.front().first;
        int mask = Q.front().second;
        Q.pop();

        if(u != N-1){

```

```

    for(int v : G[u]){
        int newMask = mask | (1<

```

Download Speed

```

#include

using namespace std;
typedef long long ll;
typedef pair pil;
const int maxN = 501;
const ll INF = 0x3f3f3f3f3f3f3f;

int N, M, p[maxN];
ll cap[maxN][maxN];
vector G[maxN];

ll bfs(int s = 1, int t = N){
    fill(p+1, p+N+1, -1);
    p[s] = -2;

    queue Q;
    Q.push({s, INF});
    while(!Q.empty()){
        int u = Q.front().first;
        ll f = Q.front().second;
        Q.pop();

        for(int v : G[u]){
            if(p[v] == -1 && cap[u][v]){
                p[v] = u;
                ll aug = min(f, cap[u][v]);
                if(v == t) return aug;
                Q.push({v, aug});
            }
        }
    }

    return 0;
}

ll maxflow(int s = 1, int t = N){
    ll flow = 0, aug = 0;
    while(aug = bfs()){
        flow += aug;
        int u = t;
        while(u != s){
            int v = p[u];
            cap[v][u] -= aug;
            cap[u][v] += aug;
            u = v;
        }
    }
    return flow;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        ll c;
        scanf("%d %d %lld", &a, &b, &c);

```

```

    G[a].push_back(b);
    G[b].push_back(a);
    cap[a][b] += c;
}
printf("%lld\n", maxflow());
}

```

Police Chase

```

#include

using namespace std;
typedef pair pii;
const int maxN = 501;
const int INF = 0x3f3f3f3f;

int N, M, p[maxN], cap[maxN][maxN];
bool vis[maxN];
vector G[maxN];

int bfs(int s = 1, int t = N){
    fill(p+1, p+N+1, -1);
    p[s] = -2;

    queue Q;
    Q.push({s, INF});
    while(!Q.empty()){
        int u = Q.front().first;
        int f = Q.front().second;
        Q.pop();

        for(int v : G[u]){
            if(p[v] == -1 && cap[u][v]){
                p[v] = u;
                int aug = min(f, cap[u][v]);
                if(v == t) return aug;
                Q.push({v, aug});
            }
        }
    }

    return 0;
}

void dfs(int u = 1){
    vis[u] = true;
    for(int v : G[u])
        if(!vis[v] && cap[u][v])
            dfs(v);
}

int maxflow(int s = 1, int t = N){
    int flow = 0, aug = 0;
    while(aug = bfs()){
        flow += aug;
        int u = t;
        while(u != s){
            int v = p[u];
            cap[v][u] -= aug;
            cap[u][v] += aug;
            u = v;
        }
    }
}

```

```

    }
}
return flow;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
        cap[a][b]++;
        cap[b][a]++;
    }
    printf("%d\n", maxflow());

    dfs();
    for(int u = 1; u <= N; u++){
        if(!vis[u]) continue;
        for(int v : G[u])
            if(!vis[v])
                printf("%d %d\n", u, v);
    }
}

```

School Dance

```

#include

using namespace std;
typedef pair pii;
const int maxN = 505;

bool used[maxN];
int N, M, K, cnt, mt[maxN];
vector G[maxN];
vector pairs;

bool kuhns(int u){
    if(used[u]) return false;

    used[u] = true;
    for(int v : G[u]){
        if(!mt[v] || kuhns(mt[v])){
            mt[v] = u;
            return true;
        }
    }

    return false;
}

int main(){
    scanf("%d %d %d", &N, &M, &K);
    for(int i = 0, a, b; i < K; i++){
        scanf("%d %d", &a, &b);
        G[b].push_back(a);
    }

    for(int i = 1; i <= M; i++){
        fill(used+1, used+N+1, false);
    }
}

```

```

        kuhns(i);
    }

    cnt = 0;
    for(int i = 1; i <= N; i++){
        if(mt[i]){
            pairs.push_back({i, mt[i]});
            cnt++;
        }
    }

    printf("%d\n", cnt);
    for(pii P : pairs)
        printf("%d %d\n", P.first, P.second);
}

```

Distinct Routes

```

#include

using namespace std;
typedef pair pii;
const int maxN = 501, maxM = 1001;
const int INF = 0x3f3f3f3f;

int N, M, p[maxN], cap[maxN][maxN];
bool vis[maxM];
vector path, F[maxN];
vector G[maxN];

int bfs(int s = 1, int t = N){
    fill(p+1, p+N+1, -1);
    p[s] = -2;

    queue Q;
    Q.push({s, INF});
    while(!Q.empty()){
        int u = Q.front().first;
        int f = Q.front().second;
        Q.pop();

        for(int v : F[u]){
            if(p[v] == -1 && cap[u][v]){
                p[v] = u;
                int aug = min(f, cap[u][v]);
                if(v == t) return aug;
                Q.push({v, aug});
            }
        }
    }

    return 0;
}

void dfs(int u = 1){
    path.push_back(u);
    if(u == N) return;
    for(pii e : G[u]){
        int v = e.first;
        int id = e.second;
        if(cap[u][v] == 0 && !vis[id]){

```

```
        vis[id] = true;
        dfs(v);
        return;
    }
}

int maxflow(int s = 1, int t = N){
    int flow = 0, aug = 0;
    while(aug = bfs()){
        flow += aug;
        int u = t;
        while(u != s){
            int v = p[u];
            cap[v][u] -= aug;
            cap[u][v] += aug;
            u = v;
        }
    }
    return flow;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back({b, i});
        F[a].push_back(b);
        F[b].push_back(a);
        cap[a][b]++;
    }

    int K = maxflow();
    printf("%d\n", K);
    for(int k = 0; k < K; k++){
        path.clear();

        dfs();
        int sz = (int) path.size();
        printf("%d\n", sz);
        for(int i = 0; i < sz; i++)
            printf("%d%c", path[i], (" \n")[i==sz-1]);
    }
}
```
